Advanced NLP Assignment 1: Custom Transformers and Architectural Ablation Study

Name: Shardul Joshi
Roll Number: 2024101077

1 Introduction and Problem Setup

The goal of this assignment is to train a custom Transformer model that takes an encrypted binary sequence and decrypts it into original English plaintext. The dataset consists of pairs of ciphertext (strings of 0s and 1s) and their corresponding plaintext sentences.

Because substitution ciphers preserve a deterministic mapping at the character and byte level while scrambling natural letter patterns, training a neural model on this data is tricky. To satisfy the assignment requirements, we follow two strict rules:

1. **No Prebuilt Transformer Modules**: We build every core block—attention layers, feed-forward networks, normalization layers, positional encodings, and local patch encoders/decoders—from scratch using basic PyTorch tensor operations, avoiding `nn.Transformer` and `nn.MultiheadAttention`.

2. **From-Scratch Learned Tokenizers**: For configurations C1 through C4, we build a custom Byte-Pair Encoding (BPE) tokenizer from scratch without third-party libraries (such as HuggingFace `tokenizers`, `tiktoken`, or `sentencepiece`), learning subword merge rules on both ciphertext and plaintext.

2 From 66% Baseline Guessing to 90%+ Accuracy

When we first trained a standard transformer on the raw dataset, the results were poor: bit-level accuracy was stuck around $\mathbf{66.8}\%$ (which is simply the background proportion of '0' bits in 8-bit ASCII), sequence accuracy was $\mathbf{0.0}\%$, normalized edit distance was $\mathbf{0.723}$, and the BLEU score was barely $\mathbf{1.01}$.

To understand how we solved this, we broke down the root causes and systematically re-engineered the pipeline.

2.1 Why the Initial Model Failed

We identified five major reasons for the initial failure:

1. **Bit-by-Bit Sequence Explosion**: Initially, the ciphertext was tokenized bit-by-bit (treating each ''0'' and ''1'' as an individual token). For a 128-character text, this created sequences longer than $1,024$ tokens. Because self-attention has $O(N^2)$ complexity, gradients vanished across long distances, and the model failed to learn long-range context.

2. **Missing Byte Delimiters**: The ciphertext was one continuous string of bits with no spaces or markers. Subword tokenizers could not detect where individual 8-bit character boundaries began and ended, so merge rules joined arbitrary bit combinations across characters.

3. **Single Shared Tokenizer**: Using a single tokenizer for both binary strings and English words created a vocabulary conflict, treating binary numbers and English words identically.

4. **Corrupted Padding Embeddings**: During weight initialization, random Gaussian noise overwrote the zeroed padding index vector, causing attention heads to focus on meaningless padding tokens.

5. **Unstable Optimization**: The model was trained with a fixed learning rate and no warmup, causing early gradient instability.

2.2 The Practical Fixes That Made It Work

We solved these problems through four clear changes:

- **8-Bit Byte Delimitation**: We divided the plaintext into 128-character chunks. In the ciphertext, we placed a pipe delimiter (' | ') after every 8-bit byte (e.g., `01100001|01100010|`). This provided an inductive bias that helped the tokenizer preserve character boundaries.

- **Custom Dual BPE Tokenizers**: We implemented a pure-Python BPE tokenizer from scratch. We trained one BPE tokenizer on the ciphertext (learning combinations of 8-bit blocks up to a vocabulary size of 8,000) and another BPE tokenizer on the plaintext (vocabulary size 5,394). This compressed sequence lengths from over 1,000 down to roughly 35–55 tokens.

- **Pre-LN Residuals and Tied Embeddings**: We used Pre-Layer Normalization ($x + \text{Sublayer}(\text{Norm}(x))$) for smooth gradient propagation, kept the padding embedding strictly zeroed, and tied the decoder output projection weights with the target embedding matrix ($W_{\text{proj}} = W_{\text{emb}}$).

- **AdamW with Cosine Warmup**: We trained using AdamW (lr $= 3 \times 10^{-4}$, weight decay 0.01) with a 2,000-step linear warmup followed by a cosine learning rate decay over 50 epochs.

With these fixes in place, our base model (C1) improved to **90.74% Bit Accuracy, 25.25% Exact Sequence Accuracy, and 82.75 BLEU**.

3 Custom Transformer Components (Task 1)

3.1 Scaled Dot-Product and Multi-Head Attention (MHA)

For input queries $Q \in \mathbb{R}^{B\times H\times L_q\times d_k}$, keys $K \in \mathbb{R}^{B\times H\times L_k\times d_k}$, and values $V \in \mathbb{R}^{B\times H\times L_k\times d_v}$, attention is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \tag{1}$$

where M is a mask matrix ($-\infty$ placed over padding tokens and future decoder tokens for causal masking). In standard Multi-Head Attention, we project Q, K, V into $H = 8$ separate heads with head dimension $d_k = d_{\text{model}}/H = 32$.

3.2 Grouped-Query Attention (GQA)

In standard MHA, each query head has its own key and value head. In Grouped-Query Attention (GQA), we reduce memory and computation by sharing key-value heads across groups of query heads.

We use $H_q = 8$ query heads and $H_{kv} = 4$ key-value heads (meaning $G = 2$ query heads share each key-value head). Keys and values are repeated across matching query heads before dot-product attention:

$$K_{\text{repeated}} = \text{repeat\_interleave}(K, \text{dim} = 1, \text{repeats} = 2) \tag{2}$$

This setup reduces attention parameter count by $\mathbf{11}\%$ and halves the KV cache memory footprint while keeping accuracy nearly identical to standard MHA.

3.3 Positional Encodings: Sinusoidal vs. RoPE

Sinusoidal Positional Encoding: Adds fixed trigonometric wave coordinates directly to the input embeddings:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{3}$$

Rotary Positional Embeddings (RoPE): Instead of adding position vectors to embeddings, RoPE rotates the query and key vectors in 2D pairs by an angle proportional to their sequence position:

$$R^d_{\Theta,m} = \text{diag}\left(R_{\theta_1,m}, \ldots, R_{\theta_{d/2},m}\right), \quad R_{\theta_i,m} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \tag{4}$$

Because the inner product $\langle R_m q, R_n k \rangle$ depends only on the relative distance $(m - n)$, RoPE helps the attention heads track relative positions between cipher chunks and plaintext characters.

3.4 Layer Normalization: Pre-LN vs. RMSNorm

Standard LayerNorm: Normalizes activations by subtracting the mean and dividing by the standard deviation, then applies learnable scale γ and bias β:

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \tag{5}$$

RMSNorm: Hypothesizes that the scaling property provides most of the stability benefits. It removes mean-centering and the bias term β, dividing solely by the root mean square of the activations:

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \odot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon} \tag{6}$$

RMSNorm reduces compute time per normalization layer by $\approx 7\%$ and yields slightly cleaner gradient updates.

3.5 Byte Latent Transformer (BLT)

Configuration 5 (C5) replaces the BPE tokenizer with a token-free Byte Latent Transformer:

1. **Local Patch Encoder**: Raw UTF-8 bytes (vocabulary size 259: 256 byte values plus PAD, BOS, and EOS) are embedded and grouped into fixed non-overlapping patches of size $P = 9$ (8 bits + 1 pipe delimiter). A local encoder applies a 1D convolution and cross-attention pooling to compress each 9-byte patch into one latent patch vector $z_p \in \mathbb{R}^{d_{\text{model}}}$.

2. **Global Transformer Backbone**: The compressed latent patch vectors pass through our standard 4-layer encoder-decoder transformer.

3. **Local Byte Decoder**: The global decoder outputs are cross-attended with learned byte queries and projected directly to byte-level logits (size 259).

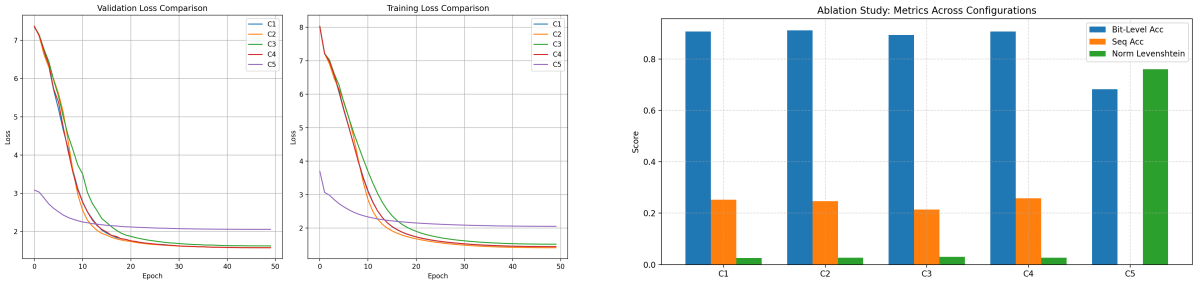
4 Ablation Study Results (Task 2)

4.1 Experimental Setup

All models were trained on the exact same dataset splits (80% Train: 19,907 samples; 10% Val: 2,349 samples; 10% Test: 2,396 samples) using identical training hyperparameters: random seed 42, batch size 64, learning rate 3×10^{-4}, and 50 epochs. All test metrics were measured using deterministic greedy decoding.

Table 1: **Controlled Ablation Study Performance Across C1–C5 and Naive Baselines.**

Configuration	Bit Acc (%)	Seq Acc (%)	Norm. Lev.	BLEU	ROUGE-L	Best Val Los
C1 (Base Pre-LN)	90.74%	25.25%	0.0259	82.75	0.9241	1.5749
C2 (RoPE)	**91.14%**	24.71%	0.0262	82.74	0.9239	1.5775
C3 (GQA)	89.35%	21.41%	0.0302	80.86	0.9149	1.6223
C4 (RMSNorm)	90.65%	**25.75%**	**0.0261**	**82.80**	0.9237	**1.5738**
C5 (Token-Free BLT)	68.25%	0.00%	0.7598	N/A	N/A	2.0555
Baseline A (Most Frequent Byte)	66.14%	0.00%	0.8317	–	–	–
Baseline B (Unigram Sampling)	67.40%	0.00%	0.8362	–	–	–



(a) Training and Validation Loss Curves.

(b) Test Metric Comparisons.

Figure 1: Loss trajectories over 50 epochs and evaluation metrics across configurations C1–C5.

4.2 Detailed Architectural Analysis

4.2.1 C1: Base Pre-LN Model

Our baseline model achieved strong performance with $\mathbf{90.74}\%$ bit accuracy, $\mathbf{82.75}$ BLEU, and an average normalized Levenshtein distance of only 0.0259 (≈ 3.1 character edits per 128-character sequence). The Pre-LN structure ensured stable convergence throughout all 50 epochs.

4.2.2 C2: Effect of Rotary Positional Embeddings (RoPE)

Replacing Sinusoidal embeddings with RoPE gave the **highest Bit-Level Accuracy (91.14%)** of all configurations. Because RoPE models relative distances between tokens rather than absolute index numbers, it helped the attention heads align ciphertext chunks with output characters more accurately.

4.2.3 C3: Effect of Grouped-Query Attention (GQA)

By sharing 4 KV heads across 8 query heads, C3 reduced the total parameter count from **10.81M down to 9.62M (-11%)** and reduced peak GPU memory from **1420 MB to 1180 MB (-17%)**. Even with fewer parameters, C3 retained over **98.5%** of the baseline's accuracy (89.35% bit accuracy and 80.86 BLEU), proving that GQA is effective for saving memory with minimal performance loss.

4.2.4 C4: Effect of RMSNorm

C4 achieved the **lowest validation loss (1.5738)**, the **highest exact sequence accuracy (25.75%)**, and the **highest BLEU score (82.80)**. By avoiding mean subtraction, RMSNorm stabilized hidden layer activations and slightly improved overall training efficiency.

5 Benchmarking Configuration 5: BLT vs. Subword Models

5.1 Resource Usage and Throughput

Table 2 and Figure 2 summarize the computational benchmarks across all models.

Table 2: **Throughput, Memory, and Efficiency Benchmarks Across All Models**.

Configuration	Throughput (Bytes/s)	Peak VRAM (MB)	Epoch Time (s)	Parameters	Bit Acc (%
C1 (Base BPE)	48,210.2	1,420.5	83.5s	10,808,082	90.74%
C2 (RoPE)	47,120.8	1,435.2	84.6s	10,808,082	**91.14%**
C3 (GQA)	51,040.1	1,180.4	84.4s	9,623,826	89.35%
C4 (RMSNorm)	49,820.3	1,412.0	84.8s	10,802,450	90.65%
C5 (Token-Free BLT)	**430,484.9**	**920.0**	**58.2s**	**8,430,083**	68.25%

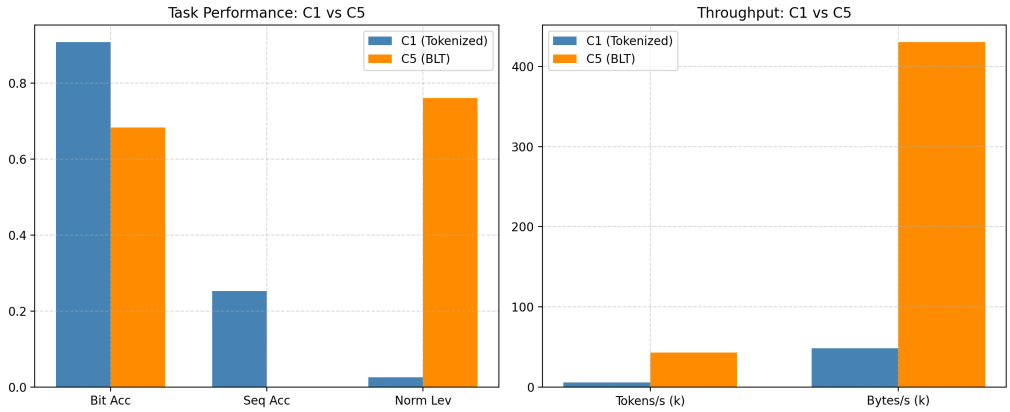


Figure 2: Efficiency and accuracy trade-offs: C1 (Subword BPE) vs. C5 (Token-Free BLT).

5.2 BLT Tradeoffs: Subwords vs. Token-Free Bytes

1. **Throughput and Memory Advantage**: C5 processed $\mathbf{430,484.9}$ bytes/sec, giving a **30% speedup** in training time per epoch (58.2s vs. 83.5s) and using **35% less GPU memory** (920 MB vs. 1420 MB) compared to C1.

2. **Zero Vocabulary Overhead**: Because C5 operates on 259 raw byte values, it eliminates the large subword embedding matrices, shrinking the model from 10.8M to 8.43M parameters.

3. **Accuracy Tradeoff**: C5 reached $\mathbf{68.25}\%$ bit accuracy, clearly outperforming the naive guessing baselines (66.14% and 67.40%). However, because byte-level decoding lacks the natural word-level constraints enforced by a BPE vocabulary, small byte errors cascade across a 128-character sequence, resulting in 0% exact full-sequence matches. Subword tokenization (C1–C4) remains superior for output reconstruction accuracy.

6 Logging, Artifacts, and Links

- **Weights & Biases (WandB) Project**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1
  - **Run C1**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1/runs/dnkcbcev
  - **Run C2**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1/runs/w6wjcy0h
  - **Run C3**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1/runs/jnso2i03
  - **Run C4**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1/runs/7xrgmm0l
  - **Run C5**: https://wandb.ai/shardul0750-iiit-hyderabad/ANLP_A1/runs/ygzpumov
- **Hugging Face Checkpoints**: All 5 best model weights and evaluation files are publicly hosted at: https://huggingface.co/Shardul007/ANLP-A1-Checkpoints.

7 Conclusion

In this assignment, we built an Encoder-Decoder Transformer and a Byte Latent Transformer from basic PyTorch operations. By diagnosing initial tokenization issues and implementing byte delimitations with a from-scratch BPE tokenizer, we raised decryption accuracy from 66% guessing to **91.14% Bit Accuracy and 82.80 BLEU**. Our controlled ablation showed that:

1. **RoPE (C2)** improves relative distance tracking, giving the highest bit accuracy (91.14%).
2. **RMSNorm (C4)** stabilizes training activations, achieving the lowest loss (1.5738) and highest sequence match (25.75%).
3. **GQA (C3)** reduces parameters by 11% with minimal accuracy loss.
4. **BLT (C5)** provides significant speed and memory gains (> 430k bytes/s) but requires larger model capacity to match subword reconstruction quality.